HTTPS and application roles reference

Use with Phase 3's certificates, Apache setup, six access cases and permission revocation for FWR-03/FWR-05 and FWT-03/FWT-05. Obtain G approval for signing/trust, H for the application and E for the matching directory change, naming targets, private values, validation and recovery. Keep the offline signer and APP1 console available. Read each command's target and privilege label before use.

The conditional profiles are OpenSSL 3.x in a POSIX-compatible shell, Ubuntu Server with Apache 2.4/www-data, and an approved Windows lab browser using its reviewed certificate store. Record exact installed versions before execution. Do not paste these commands into Windows PowerShell, Proxmox or OPNsense. Use Phase 2's Windows reference and Linux host policy for existing dependencies; this reference changes no topology or flow. Physical HP OPNsense still owns routed FF-03; APP1 host policy and Apache apply their separate checks.

Offline CA and private-key custody

Purpose and location: under G approval, use the existing approved offline signing workstation for the CA and APP1's console for its server key. Do not create a CA VM or install certificate services.

1. On the offline signer, open a terminal in a new approved owner-only directory outside every Git checkout. Use a POSIX-compatible shell and OpenSSL 3.x matching the req reference. Inspect the directory first: the following output paths must not already contain keys, certificates or serial state.

CONFIGURATION CHANGE - offline signer, signing-file owner; root not required. Set private path variables in the already approved current directory and restrict new-file permissions. There are no secret values in these assignments:

```
umask 077
LabCaKeyPath="$PWD/lab-ca.key"
LabCaCertPath="$PWD/lab-ca.crt"
LabAppCsrPath="$PWD/app.csr"
LabAppCertPath="$PWD/app.crt"
LabLeafExtensionsPath="$PWD/app-extensions.cnf"
LabSerialPath="$PWD/lab-ca.srl"
```

Check: these six paths resolve inside the approved private signing directory; outputs are new and source transfers will be reviewed. If any path exists unexpectedly, stop rather than replace it.

CONFIGURATION CHANGE - same offline signer/file owner. Create one 3072-bit encrypted CA key and a 365-day public root certificate. Keep the prompted passphrase in approved recovery custody; never supply it on the command line:

```
openssl req -x509 -newkey rsa:3072 -sha256 -days 365 \
    -keyout "$LabCaKeyPath" -out "$LabCaCertPath" -subj "/CN=HL-CYB-001 Lab CA" \
    -addext "basicConstraints=critical,CA:TRUE" \
    -addext "keyUsage=critical,keyCertSign,cRLSign"
```

Check: the key and public certificate are created without an error. Inspect the public properties and encrypted-key readability without printing the key:

READ-ONLY - offline signer, signing-file owner; no root. These show public CA constraints/fingerprint and check the encrypted key through its passphrase prompt. No replacement values beyond the defined paths:

```
openssl x509 -in "$LabCaCertPath" -noout -fingerprint -sha256 -ext basicConstraints,keyUsage
openssl pkey -in "$LabCaKeyPath" -noout -check
```

Check: CA:TRUE, certificate-signing usage, a recorded SHA-256 fingerprint and successful key check after the passphrase prompt. Store only public properties in **02 / HTTPS Certificate Configuration**. Failed key/passphrase verification blocks signing; preserve files and resolve custody without regenerating over them.

APP1 key and certificate request

Target / approval: APP1 Proxmox console, G/H scope and sudo/root. Preserve existing files and service-account details privately; do not create another CA or overwrite a key. The expected result is a private server key and a verifiable public request. On error stop transfer and use the recovery below.

1. On APP1, prepare **/etc/labapp** as a restricted directory for the application. In the conditional Ubuntu profile Apache workers use **www-data**; verify the selected package account first. Run these commands only for a new approved lab directory, not over existing owner content.

CONFIGURATION CHANGE - APP1 console, sudo/root, G/H approved scope. Create the private configuration directory:

```
sudo install -d -o root -g www-data -m 0750 /etc/labapp
```

CONFIGURATION CHANGE - APP1 console, root shell. Run `sudo -i` on its own. Confirm the root shell opened before setting its file-creation mask and paths below. Keep that shell for the key and request commands.

```
umask 077
LabAppKeyPath=/etc/labapp/app.key
LabAppCsrPath=/etc/labapp/app.csr
```

Check: the directory's owner/group/mode match the approved Apache account and the two files do not already exist. If the service account differs, correct the reviewed profile first.

CONFIGURATION CHANGE - APP1, that root shell, new approved paths. Create its 2048-bit service key and certificate signing request (CSR). The request is public signing input; it is not yet a certificate:

```
openssl req -new -newkey rsa:2048 -noenc -keyout "$LabAppKeyPath" \
    -out "$LabAppCsrPath" -subj "/CN=app.ad.lab.test"
```

Check: request subject is app.ad.lab.test and the server key is root-owned/private. The **-noenc** option intentionally leaves this service key unencrypted so the service can start; strict permissions and encrypted backup protect it. Inspect the CSR with **openssl req -in "$LabAppCsrPath" -noout -subject -verify** in the same root shell: expect the matching subject and successful request signature. On error, stop before transfer and preserve the current files.

2. Transfer only **app.csr** to the signer's LabAppCsrPath through the approved private media path. Keep the APP1 key on APP1 and the CA key on the offline signer. Compare the request name before signing; never publish either private key or its passphrase.

If it fails: preserve existing keys/request and stop before signing or trust changes. Resolve file identity, permissions, passphrase custody or the selected OpenSSL syntax; never overwrite a key to hide the error.

Leaf signing and verification

Purpose and location: on the offline signer, issue one application certificate for app.ad.lab.test; later import only the public root on approved lab clients.

1. Use the approved editor to save the following as LabLeafExtensionsPath. Extensions describe what the certificate is allowed to identify: **SAN** is its subject alternative name, and **serverAuth** means a web-server certificate.

CONFIGURATION CHANGE - offline signer, signing-file owner; no root. Exact configuration-file content, not shell commands:

```
basicConstraints=critical,CA:FALSE
keyUsage=critical,digitalSignature,keyEncipherment
extendedKeyUsage=serverAuth
subjectAltName=DNS:app.ad.lab.test
```

Check: the leaf is not a CA, permits server authentication and names exactly app.ad.lab.test. A different hostname needs review before signing.

CONFIGURATION CHANGE followed by READ-ONLY - offline signer, signing-file owner. Sign the reviewed CSR for 90 days, maintaining private LabSerialPath; then verify the chain/name and show dates/SAN. Variables are those defined in offline custody. OpenSSL x509 and verify document the options:

```
openssl x509 -req -in "$LabAppCsrPath" -CA "$LabCaCertPath" -CAkey "$LabCaKeyPath" \
    -CAcreateserial -CAserial "$LabSerialPath" -days 90 -sha256 \
    -extfile "$LabLeafExtensionsPath" -out "$LabAppCertPath"
openssl verify -CAfile "$LabCaCertPath" -verify_hostname app.ad.lab.test "$LabAppCertPath"
openssl x509 -in "$LabAppCertPath" -noout -dates -ext subjectAltName
```

Check: verification returns **OK**, the DNS SAN matches, and current time lies within the displayed validity. In the same certificate view inspect CA:FALSE, serverAuth and key usage; calculate/record remaining validity from the displayed end date and actual clock. Do not change domain time to test expiry. Failure blocks service use; do not bypass it with browser warnings.

Public certificate transfer and browser trust

Target / approval: approved private media path, APP1 root administrator and WIN1 lab administrator, G/H trust scope. Expected: only reviewed public files move; private keys remain at their owners. If fingerprints, permissions or scope differ, stop and preserve the current files/store for recovery.

1. Return **app.crt** and the public **lab-ca.crt** to APP1 through reviewed media. Place them at **/etc/labapp/app.crt** and **/etc/labapp/lab-ca.crt** with root ownership and approved public-certificate read access. Keep **/etc/labapp/app.key** root-only, mode 0600. Inspect owners/modes with the file-properties view or root **stat**; never display key content.
2. Prepare the public root and signer fingerprint for WIN1, but **test the untrusted browser first after Apache HTTPS starts**. A private/incognito window does not remove machine-wide CA trust. Use this before/after sequence on the existing WIN1 client rather than adding a VM.

3. After the untrusted rejection is recorded, on the approved WIN1 lab client open **Manage computer certificates > Trusted Root Certification Authorities > Certificates > All Tasks > Import** as administrator. Import only the public lab-ca.crt, comparing its fingerprint with the offline signer's record. Computer-wide trust must be explicitly approved; never add it to a home/production store. Microsoft describes the root store.

Check: record the one CA/one leaf, public properties, trust scope, normal trusted handshake and untrusted-client failure in **02 / HTTPS Certificate Configuration** and **03 / Test Cases**, FWT-05. Certificate-revocation services are excluded; state that limitation. Account/device revocation remains required.

If it fails: inspect SAN, chain, dates, private DNS and the selected trust store. Remove only an incorrectly imported lab root under G recovery approval; never trust an unexpected certificate to finish a test.

Synthetic pages and application accounts

Purpose and location: on APP1 under H approval, build two static pages. The application has **separate test accounts**, distinct passwords and **manually assigned roles**. AD handles Windows logon; these Apache accounts handle the web pages. There is no single sign-on or automatic AD synchronization.

1. Use the approved package/update path to make Apache 2.4 and its password utility available. An Ubuntu install can start the default HTTP site. For a new install, use the approved release-specific method to suppress that automatic start until the HTTPS-only configuration passes its checks. Stop if that method is missing. Use approved local media when offline.

CONFIGURATION CHANGE - APP1 console, sudo/root, H approval and approved package source. For the selected Ubuntu profile, install Apache and its password utility; no placeholders:

```
sudo apt install apache2 apache2-utils
sudo install -d -o root -g root -m 0755 /srv/labapp/reports /srv/labapp/maintenance
```

Check: the packages and two root-owned directories exist. After a new install, verify Apache remains stopped and TCP 80 is closed. If packages are unavailable, signatures fail or the default site starts, stop and follow the approved recovery before continuing.

2. In the approved root editor, save a small **index.html** in each directory. For **/reports/index.html**, enter a title such as **Purple Eye - Synthetic Report** and a sentence identifying fictional data. For **/maintenance/index.html**, use **Purple Eye - Synthetic Maintenance Instructions** with harmless fictional instructions. Do not insert real business information. Pages should be root-owned, readable by Apache and unwritable by ordinary app users.
3. Check **/etc/labapp/users** does not exist before the first password command. **-c creates or overwrites the whole file**; use it once only. Prompt for distinct app passwords, never AD passwords. Apache documents htpasswd's overwrite and hashing behavior.

CONFIGURATION CHANGE - APP1 console, sudo/root; the first line is DESTRUCTIVE / HIGH-RISK if users already exists. Create three bcrypt-hashed application accounts interactively, then restrict the file to administrator and Apache reads. The usernames are fixed synthetic names:

```
sudo htpasswd -B -c /etc/labapp/users ava
sudo htpasswd -B /etc/labapp/users eli
sudo htpasswd -B /etc/labapp/users uma
sudo chown root:www-data /etc/labapp/users
sudo chmod 0640 /etc/labapp/users
```

Check: all three writes succeed; inspect account names/count without copying hashes into evidence. If a write fails, preserve the old file and resolve the cause. Do not rerun **-c** to add the next person.

4. Save the group file below at **/etc/labapp/groups** using the approved root editor. Set owner/group **root:www-data**, mode **0640**. LabAdmins is infrastructure-only and does not become a fourth application account or role.

CONFIGURATION CHANGE - APP1 root editor, /etc/labapp/groups. File content only; memberships mirror the approved AD roles manually:

```
LabReaders: ava eli
LabMaintainers: eli
```

Check: ava/eli can be readers, only eli a maintainer, and uma has neither role. Record the mapping/custody in **02 / Application Accounts and Permissions**, without passwords or hashes. A directory-group change alone will not update this file.

If it fails: correct the specific spelling/membership/permissions from the saved version. Do not replace group authorization with **Require valid-user**, which would allow every authenticated account.

Apache HTTPS and group authorization

Purpose and location: in APP1's Apache configuration, expose only the two HTTPS paths, with each path checking its own group. Use the conditional Ubuntu package layout; different selected paths need review first.

1. Save the current Apache configuration privately. In **/etc/apache2/ports.conf**, retain **Listen 443** once for the TLS module and remove the plain **Listen 80** line under H approval. Do not create a duplicate 443 listener. Disable only confirmed default site links, not unknown owner configuration.
2. Save the following as **/etc/apache2/sites-available/labapp.conf** in the root editor. Verify all referenced files exist and that Apache's privileged startup can read the server key.

CONFIGURATION CHANGE - APP1 root editor, Apache 2.4 site configuration. Exact file content; no shell commands. TLS (Transport Layer Security) protects HTTPS, while group checks decide page access:

```
<VirtualHost *:443>
    ServerName app.ad.lab.test
    DocumentRoot "/srv/labapp"
    SSLEngine on
    SSLCertificateFile "/etc/labapp/app.crt"
    SSLCertificateKeyFile "/etc/labapp/app.key"
    SSLProtocol -all +TLSv1.2 +TLSv1.3
    ErrorLog ${APACHE_LOG_DIR}/labapp-error.log
    CustomLog ${APACHE_LOG_DIR}/labapp-access.log combined
    <Directory "/srv/labapp">
        Options -Indexes
        AllowOverride None
        AuthType Basic
        AuthName "HL-CYB-001"
        AuthBasicProvider file
        AuthUserFile "/etc/labapp/users"
        AuthGroupFile "/etc/labapp/groups"
        AuthzSendForbiddenOnFailure On
        Require all denied
    </Directory>
    <Directory "/srv/labapp/reports">
        Require group LabReaders
    </Directory>
    <Directory "/srv/labapp/maintenance">
        Require group LabMaintainers
    </Directory>
</VirtualHost>
```

Check: certificate/key paths match the issued files, directory listing is off, and each child requires its intended group. Apache's default AuthMerging Off lets the child requirement replace the parent denial; inspect other inherited/global settings. mod_ssl defines the TLS directives. Do not log Authorization headers, passwords or tokens.

CONFIGURATION CHANGE - APP1 console, sudo/root, selected Ubuntu profile. Enable required modules/site and disable only the reviewed default sites. If a named default site is absent, inspect existing sites rather than disabling everything. No placeholders:

```
sudo a2enmod ssl auth_basic authn_file authz_groupfile
sudo a2dissite 000-default default-ssl
sudo a2ensite labapp
sudo apachectl configtest
```

Check: the configuration check reports **Syntax OK**. An error or unexplained warning blocks the reload; inspect the reported filename/line and restore the saved site if needed. Syntax success does not prove running HTTPS.

CONFIGURATION CHANGE followed by READ-ONLY - APP1 console, sudo/root. After Syntax OK, restart to apply the reviewed listener change and inspect service/listener state. No placeholders:

```
sudo systemctl restart apache2
sudo systemctl status apache2 --no-pager
sudo ss -lntp
```

Check: apache2 is active, TCP 443 listens and plain HTTP/extra application listeners do not. If it fails, inspect the service error and saved configuration; do not expose another port to make it work.

Certificate browser test and six-case evidence

Target / approval: WIN1 lab browser and APP1 administrator log view, G/H-approved validation. Keep a working HTTPS listener and effective host/group policy as positive controls. Record failures before recovery; a browser or network failure alone cannot establish a role denial.

1. On WIN1, confirm app.ad.lab.test resolves correctly, then open **https://app.ad.lab.test/reports/** before importing the root. Expect an untrusted-issuer rejection; do not click through. Record this FWT-05 subcheck, then perform the approved public-root import in public trust and open the same name again. Expect normal HTTPS without a warning.
2. Use a separate clean browser context for each synthetic app account. Close all private windows between users so Basic authentication credentials do not carry into another case. Enter valid **application** credentials through the browser prompt, not AD passwords.

Table 1. Certificate browser test and six-case evidence

Case	App account	Page	Expected result
C01	ava	/reports/	Allow: synthetic report, normally HTTP 200
C02	ava	/maintenance/	Deny: normally HTTP 403
C03	eli	/reports/	Allow: synthetic report, normally HTTP 200
C04	eli	/maintenance/	Allow: instructions, normally HTTP 200
C05	uma	/reports/	Deny: normally HTTP 403
C06	uma	/maintenance/	Deny: normally HTTP 403

3. In the browser developer tools **Network** tab, reload the exact page and inspect the request status. Compare it with Apache's access/error logs and the group file. A wrong password/401 does not demonstrate authenticated role denial. Record the six outcomes once in **03 / Test Cases**, FWT-03.04-.09, and link them from **02 / Six Application Access Tests**. Keep a successful eli maintenance browser open for revocation.

If it fails: check DNS/host rules first for connection failure; check chain/name/dates for HTTPS failure; check the authenticated user, exact group spelling and inherited authorization for a wrong decision. Restore the saved site, syntax-check and restart only under H recovery scope. Preserve failure evidence before fixing it.

Application permission revocation

Purpose and location: on APP1 and DC1 under H/E scope, remove eli's maintenance role. This demonstrates revocation: access that worked before the change must be denied afterward.

1. Record the change time using the separate observer clock. Preserve the approved private group-file backup. In APP1 **/etc/labapp/groups**, change the maintenance line to **LabMaintainers:** with no member. Leave **LabReaders: ava eli** unchanged; save with the same restricted ownership/mode.
2. On DC1 **AD Users and Computers > LabMaintainers > Properties > Members**, remove **eli**. Record both changes. Apache reads its own group file; Windows logon/token refresh does not revoke an application role.
3. In the already authenticated eli browser, reload **/maintenance/** with cache disabled in the developer Network panel; ensure a fresh request went to the server. Then use a fresh browser context and valid eli app credentials for another maintenance request. Both must deny, normally 403. Reading an already downloaded page from cache is not a new authorization decision.
4. Compare both requests with the changed app group file and correlated Apache denial records. Record actual elapsed time, request times and any cache/session treatment in **03 / Test Cases**, FWT-03.10-.13. Summarize in **02 / Access Revocation**. Do not claim a timing target that the assignment does not set.

Check: the formerly allowed maintenance access is now denied in both contexts. Restore eli's membership only if the approved test/recovery scope explicitly permits it, and record the restoration; do not silently change expected outcomes for later tests.

If it fails: verify the edited file is the effective AuthGroupFile, the browser made a server request, inherited configuration is unchanged and no broad rule grants access. Keep the failure; do not delete the account or disable the whole site merely to produce a denial.

The reader line must remain **LabReaders: ava eli**. After both maintenance denials, make a new eli **/reports/** request and confirm HTTP 200 against its access-log entry. Keep the standard AD identity usable. FWT-03.10 owns manual role/password custody comparison; .11 records exact app/AD removal and cache treatment; .12/.13 own existing/fresh requests and the separate-clock elapsed-time observation. Phase 2's .01-.03 and all six .04-.09 remain required; this procedure alone cannot pass the whole test. There is no revocation-time SLA.

File and certificate inspection

Target / approval: APP1 console, approved G/H root administrator; read-only inspection of existing files. Expected: three application names, protected files and matching public key material. Do not copy passwords/hashes or private-key content into evidence. On mismatch stop service use and reconcile the saved reviewed files; never add a permissive authorization rule.

READ-ONLY - APP1 console, sudo/root; selected Ubuntu paths. Check owners/modes and usernames only, then compare the two SHA-256 public-key digests. No private key is printed; identical digests establish that the leaf belongs to the server key:

```
sudo stat -c '%U:%G %a %n' /etc/labapp /etc/labapp/app.key /etc/labapp/app.crt \
    /etc/labapp/lab-ca.crt /etc/labapp/users /etc/labapp/groups
sudo cut -d: -f1 /etc/labapp/users
sudo openssl pkey -in /etc/labapp/app.key -pubout -outform DER | openssl dgst -sha256
openssl x509 -in /etc/labapp/app.crt -pubkey -noout |
    openssl pkey -pubin -outform DER | openssl dgst -sha256
```

Check: directory root:www-data 0750; key root-owned 0600; users/groups root:www-data 0640; public certificates root-owned with approved read access. The only application names are ava, eli, uma. Public pages are root-owned and Apache-readable, with no app-user write access. If a command errors, do not use its digest as a successful comparison.

READ-ONLY - offline signer, signing-file owner, the same private path variables. Supplement the original chain/date checks with explicit server-purpose and extension inspection. No changes or private values are required:

```
openssl verify -CAfile "$LabCaCertPath" -purpose sslserver \
    -verify_hostname app.ad.lab.test "$LabAppCertPath"
openssl x509 -in "$LabAppCertPath" -noout -dates \
    -ext basicConstraints,keyUsage,extendedKeyUsage,subjectAltName
openssl x509 -in "$LabCaCertPath" -noout -dates
```

Check: verification returns OK; leaf CA:FALSE, digitalSignature/keyEncipherment, serverAuth and DNS SAN app.ad.lab.test; root CA:TRUE/keyCertSign/cRLSign from the original root inspection. Both certificates are currently valid. The leaf's 90-day end must fit within the root's remaining 365-day validity; an older root cannot automatically issue another full 90-day leaf. Record actual UTC time, both end dates and remaining validity explicitly; stop signing/use when that constraint cannot be met. Do not alter domain time or silently extend approved lifetimes.

Recovery and version notes

Targets / approval: offline signer, APP1 root console and only the explicitly approved lab-client store, within the named G/H/E recovery scope. Before any write preserve existing signing serial state, Apache files/site links/ports, group/password files, owners/modes, public trust fingerprints and private backups in new owner-only locations. Preserve encrypted off-compute recovery custody; never overwrite the sole backup. Do not restore an application permission as an unrecorded cleanup.

- **Signing/key failure:** stop issuance and preserve the existing request/key/serial files. Confirm target directory, passphrase custody, permissions, CSR signature and supported options. Recover only an identified intact approved copy. A compromised key or required replacement needs a separate owner plan; this lab has no CRL/OCSP publication or certificate-revocation infrastructure. The presence of cRLSign in the CA profile does not provide a revocation service.

- **Trust/transfer failure:** compare the public root fingerprint through the approved independent record and verify the leaf/key/name/chain/dates. At WIN1 **Manage computer certificates > Trusted Root Certification Authorities > Certificates**, remove only the exact incorrectly imported lab root under G recovery approval. Inspect current-user/browser-specific stores if applicable; incognito mode does not remove trust. Never change a home/production store or click through a warning. Repeat the untrusted-before-import sequence only with approved lab-root removal and a still-functional service.
- **Application failure:** at APP1 console restore only the saved site/ports/module links and file permissions that this change owned. Run the same configuration check; restart only after Syntax OK, then inspect service, TCP 443 and lack of HTTP/extra listeners. If the saved version exposed HTTP, retain a stopped/non-exposed service until an approved safe configuration exists. Recheck affected role decisions and the authorized final membership state; an account deletion or stopped service cannot prove role revocation.
- **Version differences:** verify OpenSSL req, x509 and verify against the installed 3.x release. The reviewed profile retains one RSA-3072 encrypted 365-day root and one APP1-generated RSA-2048 key/90-day leaf. OpenSSL's -noenc is intentional only for the private service key. Apache 2.4's group provider, AuthMerging and AuthzSendForbiddenOnFailure preserve group decisions and authenticated 403s. Inspect effective Directory/Location/global overrides and loaded modules; no Require valid-user fallback, AD module, SSO bridge or extra app service is included.

Record operator/date/version, expected/actual status, exact request times, private evidence IDs and second observations in **03 > Test Cases**. Summarize in the named **02** sections; put unresolved failures in **04 > Issue Log**. Keep raw logs, credentials, keys, actual addresses and private working records outside Git. Missing execution is NOT TESTED, never a fabricated PASS.